

FIT2109 Computer Science Workshop

Week 7: Command-Line Environment

Zahra Mirzamomen

Department of Software Systems and Cybersecurity

Faculty of IT, Monash University

From using the shell to making it yours

Weeks 1–2 gave you the commands	Week 7 gives you the environment they run in
Navigate paths, manage files and permissions	Dotfiles: where settings live, and which file each shell reads
Chain commands with pipes and redirection	Command lookup: which program a name actually resolves to
Variables, quoting and exit status	\$PATH and exported variables you control
Write a script and run it with <code>./script.sh</code>	Shell customisation: aliases, functions and your own prompt
Retype each command as you need it	History you can search, and configure so it is not lost
SSH to a server and work there	Process control and sessions that outlive a dropped connection

Every part of today answers a frustration you have had

You have hit this	Today's answer
"My alias works in one terminal but not another."	Startup files — which file each kind of shell reads
"It says command not found, but the script is right there."	<code>\$PATH</code> and command lookup
"I keep retyping the same long command."	Aliases, functions, and searching your history
"I closed the laptop and lost a forty-minute job."	Job control, <code>nohup</code> , and <code>tmux</code>

Dotfiles: where your settings live

A dotfile is a config file whose name starts with a dot

The dot only makes it hidden: `ls` skips it unless you run `ls -a`. Otherwise it is ordinary text you can open in any editor.

They do three jobs	Files
Set up your shell when it starts	<code>~/ .bashrc</code> , <code>~/ .bash_profile</code>
Change how the command line behaves	<code>~/ .inputrc</code>
Configure individual programs	<code>~/ .vimrc</code> , <code>~/ .gitconfig</code>

They make a setup repeatable: copy them to a new machine, or keep them in a Git repository, and you get your environment back instead of rebuilding it from memory.

A shell reads different files depending on how it started



Interactive Login Shell

Started when you log in.
Reads system-wide and user startup files.



1. System login process
(e.g., getty, SSH login)



2. Reads login startup files
/etc/profile, ~/.bash_profile,
~/.profile



3. You get a shell prompt
Environment is set up

Examples



ssh user@host

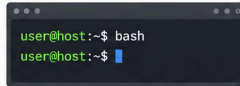


Logging in on
a virtual console



Interactive Non-Login Shell

Started from an existing shell.
Does not read login startup files.



1. Started by a user
(from an existing shell)



2. Does not read login startup files
May read ~/.bashrc (depending
on shell)



3. You get a new shell prompt
Same environment (mostly)

Examples



bash
in a terminal



Opening a new tab
in a terminal emulator



Non-Interactive Shell

Used to run commands or scripts.
No user interaction.



1. Started to run a command
or script



2. Does not read interactive
startup files (~/.bashrc)
Only non-interactive files
(if any)



3. Runs the command/script
and exits

Examples



./script.sh



cron jobs, systemd
tasks, CI/CD jobs

Which file each kind of shell reads

Shell	You get one by	It reads
Interactive login	SSH; a new macOS Terminal window	/etc/profile, then the first of ~/.bash_profile, ~/.bash_login, ~/.profile
Interactive non-login	A new tab on Linux	~/.bashrc
Non-interactive	Running a script	nothing – unless BASH_ENV is set, which normally it is not

Ask the shell you are in what kind it is

```
ps -p $$ -o comm= # which shell is running: bash, zsh, ...
shopt -q login_shell && echo login || echo non-login
[[ $- == *i* ]] && echo interactive || echo non-interactive
```

shopt, source, and the login-shell fix

1 · shopt — ask Bash about its own options

Bash keeps a list of named behaviours, each either on or off. `shopt` on its own prints the whole list, `shopt name` reports just one, and `shopt -s name / -u name` switch one on or off. One of those names tells you which kind of shell you are in: `shopt login_shell` prints `on` for a login shell and `off` for anything else.

2 · source — apply an edit to the shell you are already in

Startup files are read once, at startup, so editing `~/.bashrc` does nothing to a terminal already open. `source ~/.bashrc` (or `. ~/.bashrc`) re-runs its lines *in this shell*, so the aliases, functions and variables are there afterwards. Running `./~/.bashrc` instead would execute them in a separate process that exits and takes them with it.

3 · Make a login shell read ~/.bashrc as well

```
# in ~/.bash_profile — the file a login shell does read
if [ -f ~/.bashrc ]; then . ~/.bashrc; fi
```

Demo 1: which startup file actually ran

#	Step	Purpose
1	<code>echo 'echo BASHRC' >> ~/.bashrc</code>	Add a marker line to each candidate file
2	<code>echo 'echo PROFILE' >> ~/.profile</code>	The login-shell file gets its own marker
3	<code>bash</code>	A second Bash started <i>inside</i> the one you are using: interactive, but nothing logged you in, so not a login shell — only BASHRC prints
4	<code>bash --login</code>	Login shell: PROFILE prints instead
5	<code>bash demo.sh</code>	A script prints no marker at all
6	<code>shopt login_shell</code>	Confirm the kind of shell in each case

Then add the sourcing block to `~/.bash_profile` and repeat step 4 — BASHRC now prints too.

Other programs read dotfiles too — and why that works

The convention

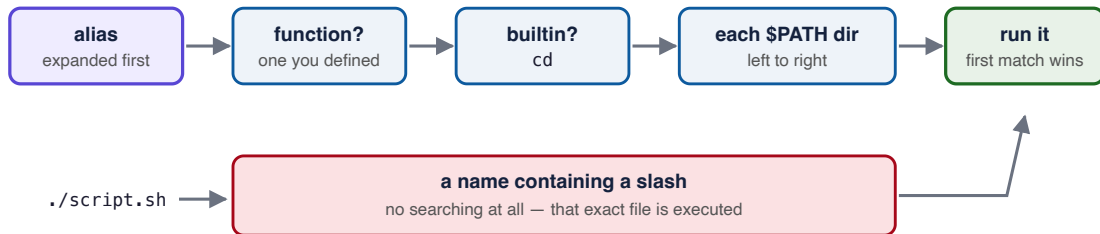
Nothing coordinates this. Each program, at its own startup, looks in your home directory for a file named after itself and reads it if it exists. The `rc` means "run commands".

File	Belongs to	Written in
<code>~/.inputrc</code>	Readline (Bash's line editor)	<code>set editing-mode vi</code>
<code>~/.vimrc</code>	Vim	<code>set number</code>
<code>~/.gitconfig</code>	Git, via <code>git config --global</code>	<code>[user] name = ...</code>

Two consequences: each file uses its own program's syntax, not shell syntax — and each is read at *that program's* startup, so editing `~/.vimrc` affects the next Vim you open, not one already running.

How Bash decides what a command name means

a name with no slash



Lookup stops at the first match, so a forgotten alias or function can silently shadow a real program.

Asking what a name actually resolves to

Command	What it tells you	What it can see
<code>type -a name</code>	Every match, in priority order, and what kind each one is	alias, function, builtin, file
<code>command -v name</code>	The single thing that would run. If nothing would, it prints nothing and exits with a non-zero status, so a script can test it	alias, function, builtin, file
<code>which name</code>	A path found by searching <code>\$PATH</code>	files only

which is the one that misleads you

`which` is not part of Bash. It is a separate program — its own executable file, `/usr/bin/which`, that your shell launches as a child process. A child process cannot look inside its parent, so all it can do is search `$PATH`: it never sees your aliases, functions or builtins. Hence `which cd` answers `/usr/bin/cd` while typing `cd` runs Bash's builtin.

command -v is also how a script checks a tool exists

Guard before you use it

```
if command -v curl >/dev/null 2>&1; then
    echo "curl is available"
else
    echo "curl is not installed or not on PATH"
fi
```

Why the redirect

`command -v` prints the path it finds. Sending output to `/dev/null` discards it, leaving only what `if` uses: the exit status, 0 for found and 1 for not.

Why not which here

`command -v` is built into the shell and specified by POSIX, so it behaves the same everywhere. `which` is an external program whose output and exit status vary between systems.

Shell Initialisation and Startup Files

Open the Week 7 handout on **Ed Lessons** and work through Activity 1 in your group of 3–4.

Demo 2: \$PATH, lookup, and a command of your own

#	Step	Purpose
1	<code>echo "\$PATH"</code>	See the directory list, in search order
2	<code>type -a cd && which cd</code>	The builtin versus the file that <code>which</code> reports
3	<code>mkdir -p ~/bin && vim ~/bin/hello</code>	Write a one-line script of your own
4	<code>chmod u+x ~/bin/hello; hello</code>	Executable, but still not found by name
5	<code>export PATH="\$HOME/bin:\$PATH"</code>	Now <code>hello</code> runs from anywhere
6	<code>export PATH="\$HOME/bin"</code>	Omitting <code>:\$PATH</code> breaks even <code>ls</code>

Step 6 is undone by opening a new terminal — the `export` only ever applied to that one shell.

Aliases: a short name for a longer command

Define, list, remove

```
alias ll='ls -lah' # define one
alias # list every alias currently defined
unalias ll # remove it
```

The shell replaces the name with its expansion *before* running the line — which is what both limits below follow from.

It cannot take an argument

Expansion is textual: the name is replaced by its text, so anything you type after it lands at the *end* of the resulting command, not where you meant it to go.

A script never sees your aliases

Two reasons: a script's shell does not read `~/.bashrc`, and alias expansion is switched off in non-interactive shells anyway. Aliases are for the keyboard; scripts need functions.

The prompt: PS1 as an information display

A template: escapes are substituted, everything else is literal

```
PS1='[\A] \u@\h:\w jobs=\j\$ ' # in ~/.bashrc
[14:32] alice@ubuntu:~/projects jobs=1$ # the prompt you get
# brackets, @ and : are literal; jobs=1 warns work is running
```

Esc	Shows	Esc	Shows
\u	Username	\j	Number of jobs this shell is managing
\h	Hostname, up to the first dot	\A	Current time, 24-hour HH:MM
\w	Working directory, ~ for home	\\$	\$ for a normal user, # for root
\W	Just the last part of it	\n	A newline, so a long prompt gets its own line

It is redrawn before every command, so keep it cheap: these escapes cost nothing, calling other programs does.

Demo 3: a .bashrc worth keeping

#	Step	Purpose
1	<pre>export PATH="\$HOME/bin:\$PATH"</pre>	Make the PATH change from Demo 2 permanent
2	<pre>alias ll='ls -lah'</pre>	Add aliases to ~/.bashrc, then reload with <code>source</code>
3	<pre>alias</pre>	List everything currently defined
4	<pre>mkcd() { mkdir -p "\$1" && cd "\$1"; }</pre>	An alias cannot do this; a function can
5	<pre>PS1='[\A] \u@\h:\w jobs=\j\\$ '</pre>	Set the prompt and see it change
6	<pre>ls() { echo oops; }; type -a ls</pre>	Shadow a real command, then <code>unset -f ls</code>

PATH Resolution, Environment Variables, and Dotfiles

Open the Week 7 handout on **Ed Lessons** and work through Activity 2 in your group of 3–4.

History is a setting, not a fixed behaviour

Recall instead of retyping

`Ctrl-R` then type: the shell searches backwards and shows the best match as you go. Press it again for an older match, Enter to run, or the Right arrow to edit first.

!! is the last command; !\$ is its last argument.

Worth putting in ~/.bashrc

```
HISTSIZE=10000
HISTFILESIZE=20000
HISTCONTROL=ignoreboth
shopt -s histappend
```

Without histappend, the last shell to close wins

It is off by default. Each shell *overwrites* the history file as it exits, so with several terminals open, everything typed in the others is lost.

Demo 4: searching and keeping your history

#	Step	Purpose
1	<code>history tail -20</code>	Numbered list of what this shell remembers
2	<code>Ctrl-R</code> then <code>"grep"</code>	Find a buried command without retyping it
3	Press <code>Ctrl-R</code> again	Step back to an older match
4	Press the Right arrow	Edit the match instead of running it
5	<code>mkdir /tmp/x; ls !\$</code>	Reuse the previous command's last argument
6	<code>shopt histappend</code>	Show it is off, then set it and explain the loss

Two terminals closing one after the other show what `histappend` prevents.

Three levels of "keep it running"

I want to...	Use
Get my prompt back while a command keeps running in this shell	<code>&</code> · <code>Ctrl-Z</code> · <code>bg</code> · <code>fg</code>
Keep one command alive after this shell closes	<code>nohup</code> · <code>disown</code>
Keep a whole session alive across a dropped connection	<code>tmux</code>

The words the rest of this depends on

A running program is a **process**, identified by a number called its **PID**. **Foreground** means it holds your terminal; **background** means it runs while you keep typing. `jobs` lists only what *this* shell started, while `ps` sees every process on the machine.

Workflow: signals, and what to reach for first

You do	Signal	What it means
Ctrl-C	SIGINT	"Please stop." The program can catch it and clean up.
Ctrl-Z	SIGTSTP	"Please pause." This is what suspends a job.
kill %1	SIGTERM	The default. A polite request it can still catch.
kill -9 %1	SIGKILL	Cannot be caught. The process gets no chance to tidy up.

Plain kill first; -9 only when it is ignored

`kill` does not destroy anything itself — it delivers a signal and the process decides how to react. `SIGKILL` skips that, which is how you end up with stale lock files and half-written output.

Demo 5: backgrounding, inspecting, signalling

#	Step	Purpose
1	<code>sleep 300 & then jobs</code>	Start in the background; note job number and PID
2	<code>sleep 300</code>	Now in the foreground — the prompt is gone
3	Press <code>Ctrl-Z</code>	Suspend it; <code>jobs</code> shows Stopped
4	<code>bg %1 then fg %1</code>	Resume in background, then pull back to foreground
5	<code>ps aux</code>	Every process, not just this shell's jobs; STAT reads T while suspended, S once resumed
6	<code>kill %1 then kill -9 <pid></code>	Polite request first, force only if ignored

Finish with `nohup sleep 300 & and disown`, then close the terminal and find the process still alive with `ps`.

Activity 3

10 min

Customising the Shell Productively

Open the Week 7 handout on **Ed Lessons** and work through Activity 3 in your group of 3–4.

Sessions, windows and panes

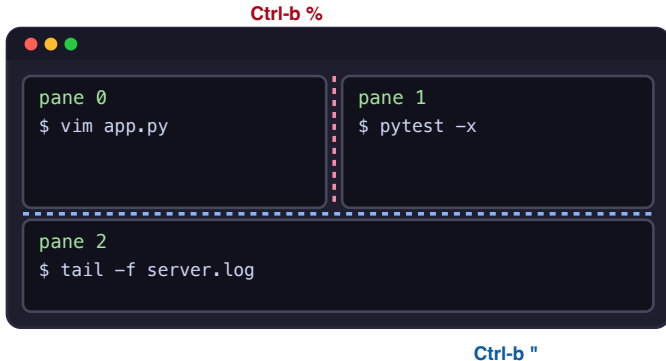
The three levels

A **pane** runs one program.

A **window** is a screenful of panes.

A **session** is named, holds windows, and is what you detach from.

The session belongs to tmux, not to your terminal window, so closing the window does not end it.



Workflow: a session that survives the connection

From the shell

```
tmux new -s work      # create and enter a named session
tmux ls               # which sessions exist, and who is attached
tmux attach -t work   # pick up exactly where you left off
tmux new -A -s work   # attach if it exists, otherwise create it
```

Inside tmux — press the prefix (Ctrl-b), release, then the key

d	Detach, leaving work running	c	New window
%	Split the pane left / right	n · p	Next · previous window
"	Split the pane top / bottom	z	Zoom one pane, again to restore
arrows	Move between panes	?	List every binding

Demo 6: detach, disconnect, reattach

#	Step	Purpose
1	<code>tmux new -s work</code>	Create a named session and enter it
2	<code>Ctrl-b %</code> then <code>Ctrl-b "</code>	Build the three-pane layout from the diagram
3	<code>Ctrl-b</code> arrows · <code>Ctrl-b z</code>	Move between panes, zoom one, restore it
4	<code>sleep 600</code>	Leave visible work running in a pane
5	<code>Ctrl-b d</code> then <code>tmux ls</code>	Detach, then confirm the session is still alive
6	<code>tmux attach -t work</code>	Everything is exactly as it was left

Closing the window entirely and reattaching from a new one makes the point.

Activity 4

10 min

Job Control and Persistent Sessions

Open the Week 7 handout on **Ed Lessons** and work through Activity 4 in your group of 3–4.

Four decisions to take away

Question	Answer
Where does this setting go?	Interactive shells read <code>~/.bashrc</code> . Login shells read <code>~/.bash_profile</code> , so source <code>~/.bashrc</code> from it. Scripts read neither.
Alias or function?	A function as soon as an argument or a decision is involved.
Which signal?	Plain <code>kill</code> first; <code>-9</code> only when it is ignored.
How much must survive?	One command needs <code>nohup</code> . A whole session needs <code>tmux</code> .

Everything today was Bash. The ideas carry across to `zsh`; the file names do not.

Quiz time!

Head to **Moodle**, under **Week 7**, and have a go at the quiz now while it is all still fresh.

Next week

Debugging: Systematic Bug Hunting